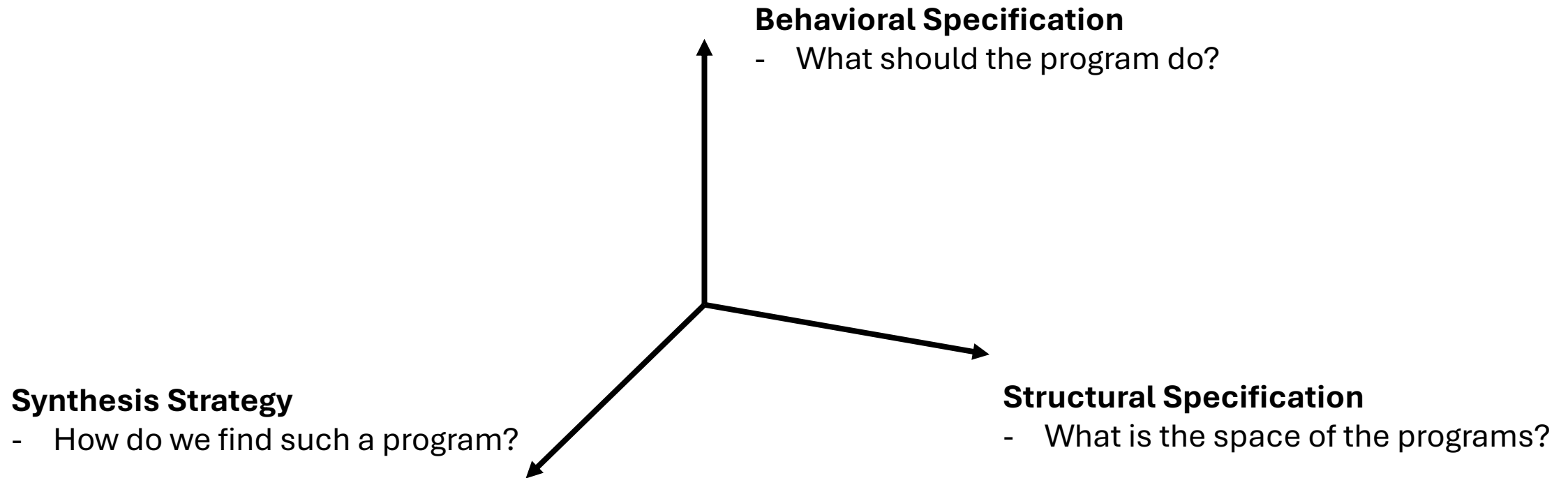


Machine Programming

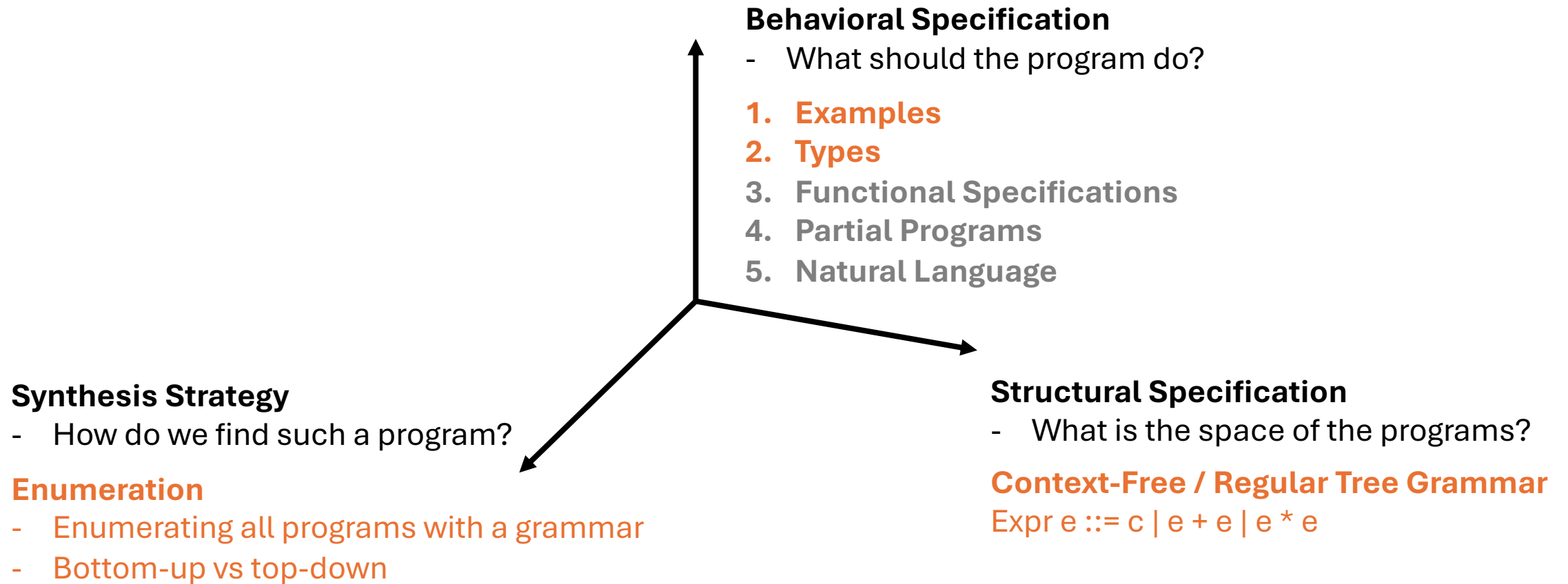
Lecture 4 – Functional Specifications for Synthesis

Ziyang Li

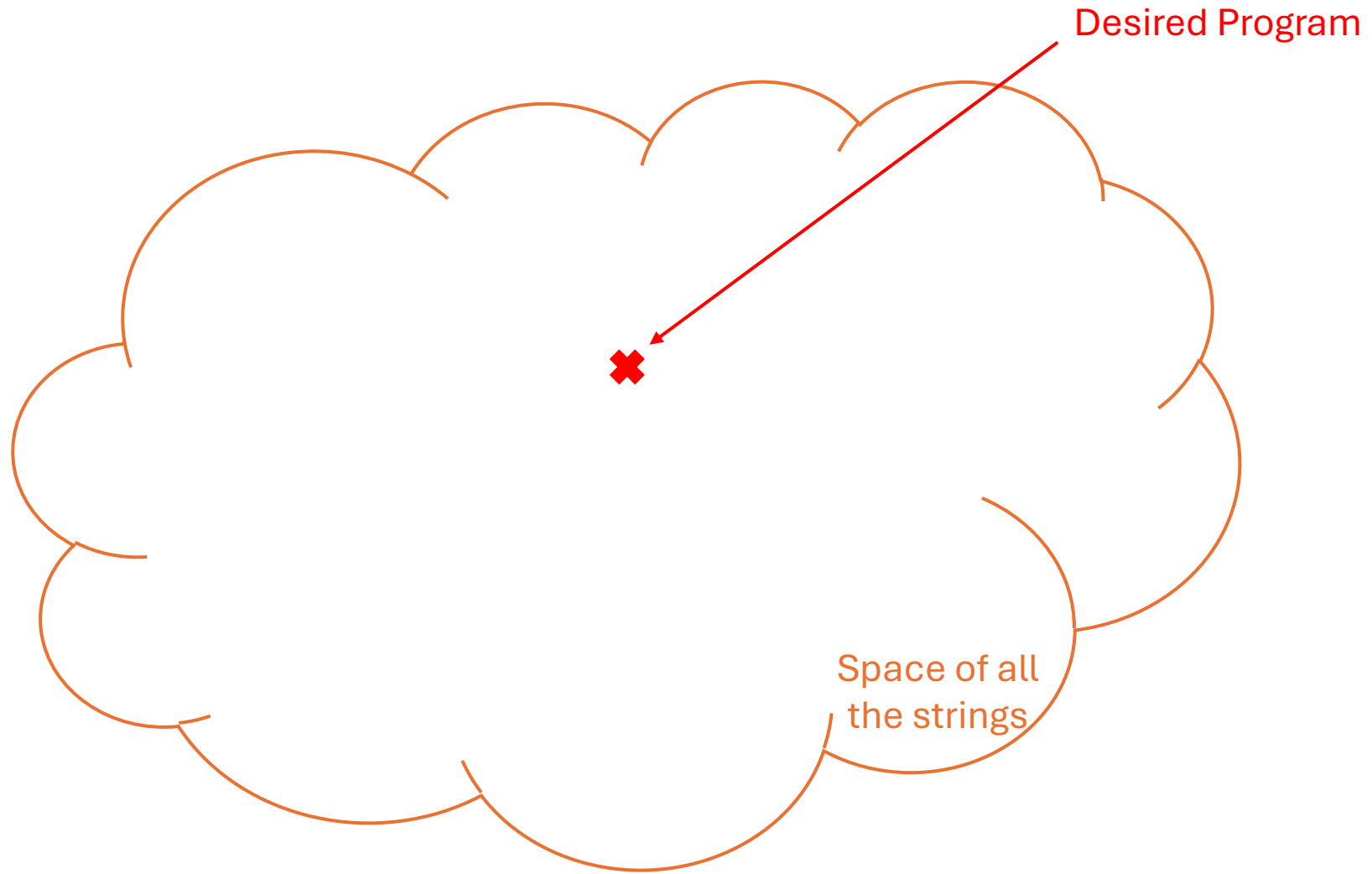
Dimensions in Program Synthesis



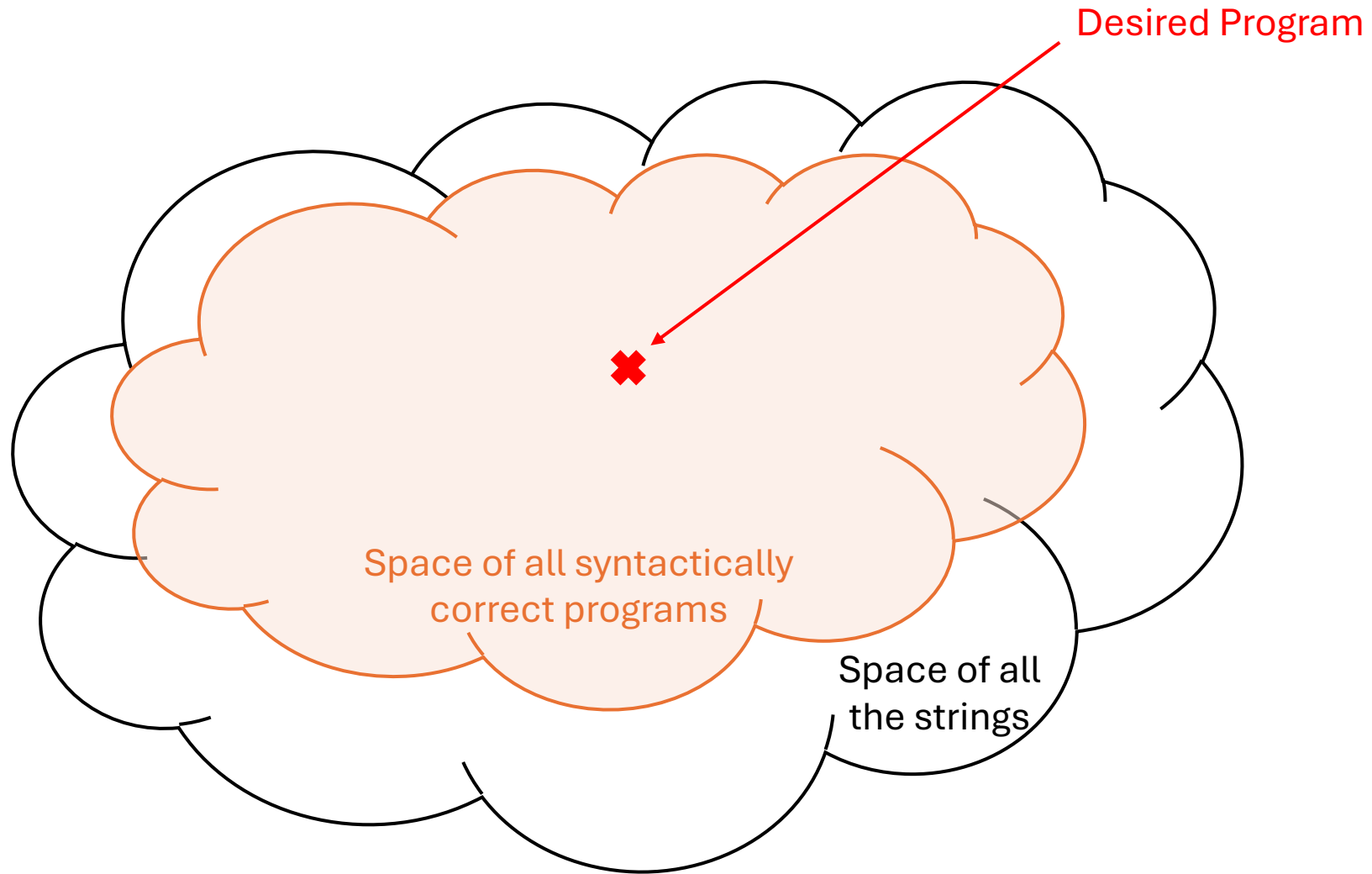
The Course So Far



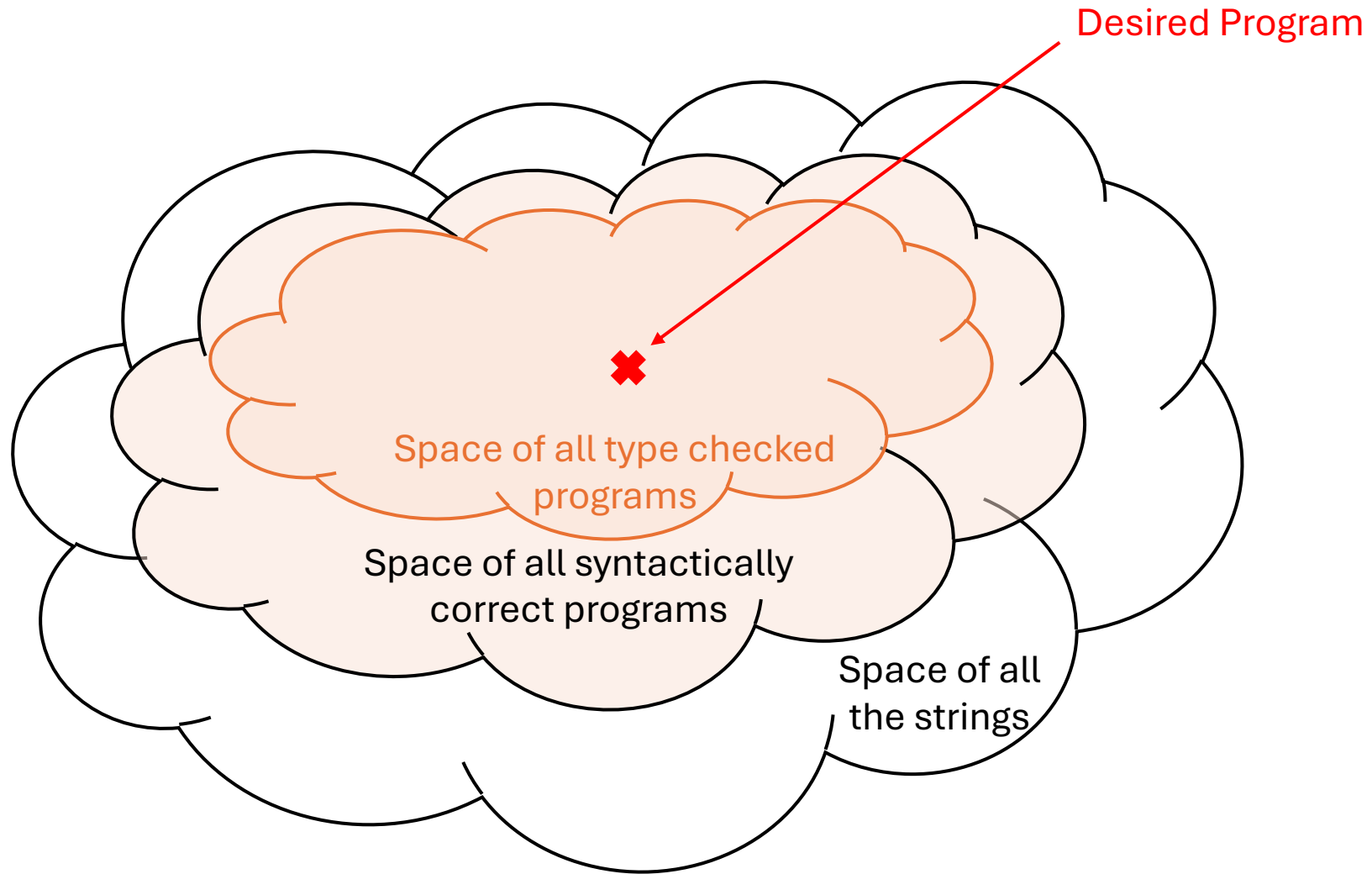
High Level Picture



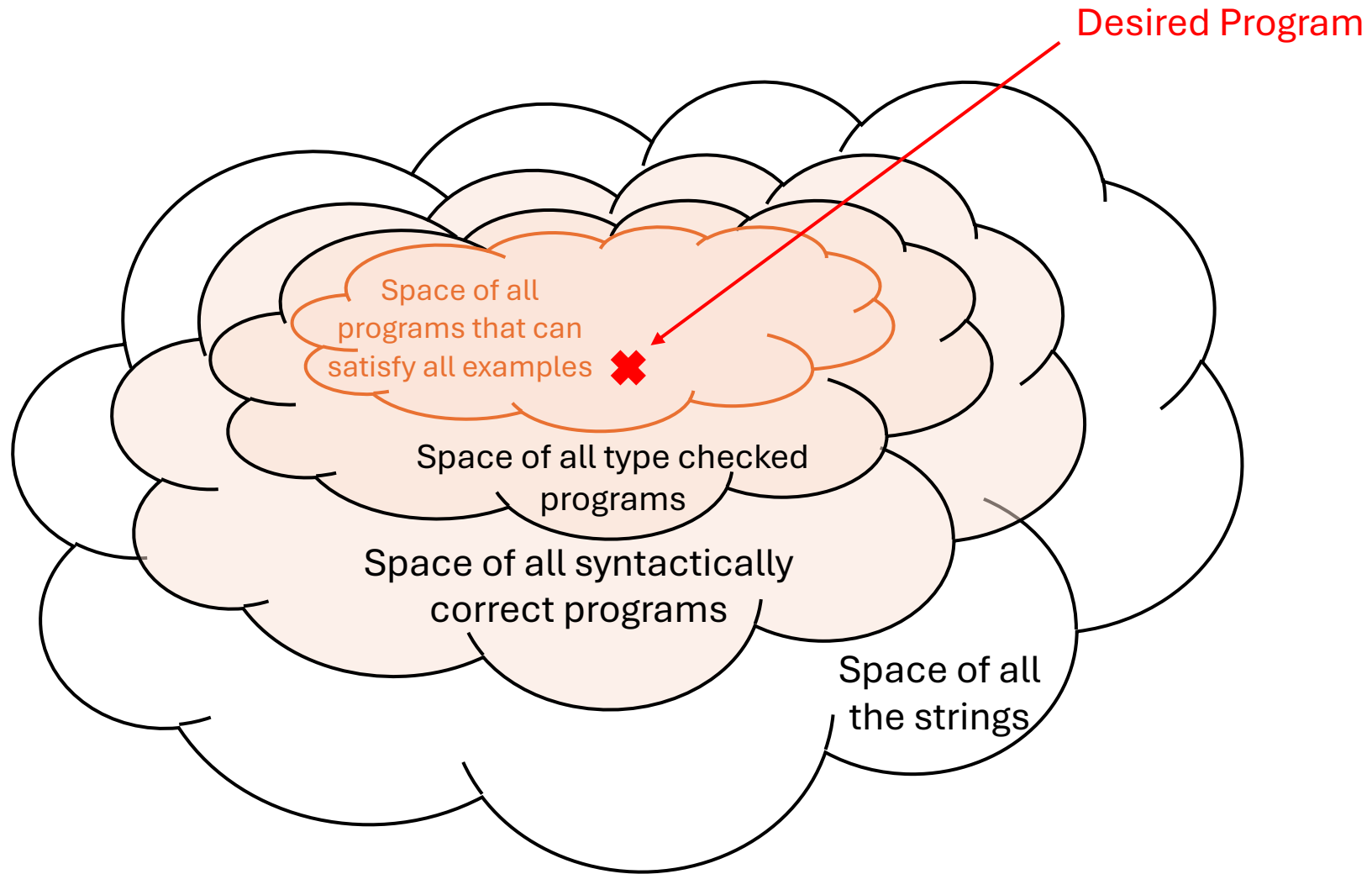
High Level Picture



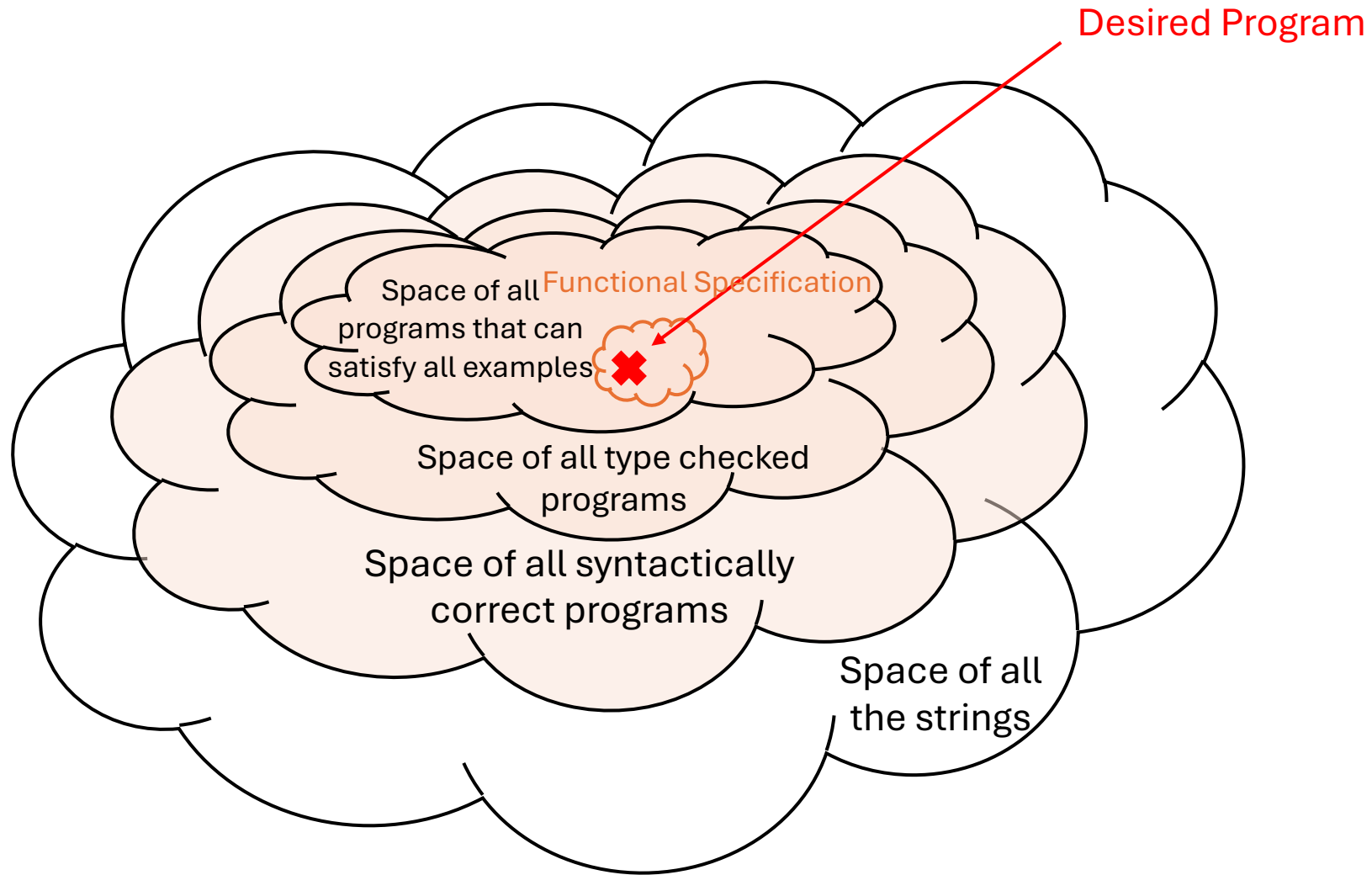
High Level Picture



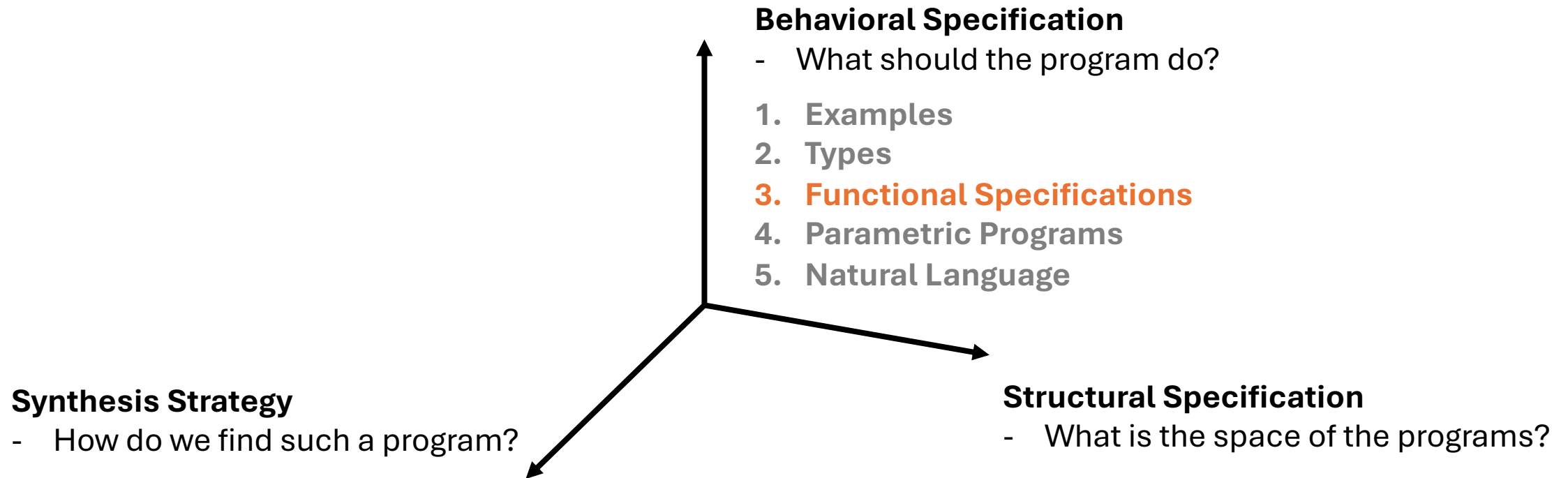
High Level Picture



High Level Picture



Today



Is Examples Enough?

$F([3, 2, 1]) = [1, 2, 3]$

$F([2, 1]) = [1, 2]$

$F([]) = []$

Is Examples Enough?

$$F([3, 2, 1]) = [1, 2, 3]$$

$$F([2, 1]) = [1, 2]$$

$$F([]) = []$$

What is F ?

Is Examples Enough?

`reverse( [3, 2, 1] ) = [1, 2, 3]`

`reverse( [2, 1] ) = [1, 2]`

`reverse( [] ) = []`

What is **F**?

Is Examples Enough?

`sort`([3, 2, 1]) = [1, 2, 3]

`sort`([2, 1]) = [1, 2]

`sort`([]) = []

What is `F`?

Is Examples Enough?

`reverse`([3, 2, 1]) = [1, 2, 3]

`reverse`([2, 1]) = [1, 2]

`reverse`([]) = []

`sort`([3, 2, 1]) = [1, 2, 3]

`sort`([2, 1]) = [1, 2]

`sort`([]) = []

What is **F**?

Is Type Enough?

```
reverse : List[int] -> List[int]  
reverse([3, 2, 1]) = [1, 2, 3]  
reverse([2, 1]) = [1, 2]  
reverse([]) = []
```

```
sort : List[int] -> List[int]  
sort([3, 2, 1]) = [1, 2, 3]  
sort([2, 1]) = [1, 2]  
sort([]) = []
```

Functional Specifications

$\forall x, y \in \text{List}[\text{Int}], \text{reverse}(x) = y \Rightarrow$

$$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 1\}, x_i = y_{\text{len}(y)-1-i}$$

`reverse([3, 2, 1]) = [1, 2, 3]`

`reverse([2, 1]) = [1, 2]`

`reverse([]) = []`

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$$

`sort([3, 2, 1]) = [1, 2, 3]`

`sort([2, 1]) = [1, 2]`

`sort([]) = []`

Functional Specifications

$\forall x, y \in \text{List}[\text{Int}], \text{reverse}(x) = y \Rightarrow$

$$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 1\}, x_i = y_{\text{len}(y)-1-i}$$

1. Input and output has the same length n ;
2. The i -th element in x is the same as $(n - 1 - i)$ -th element in y

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$$

```
reverse([3, 2, 1]) = [1, 2, 3]
reverse([2, 1]) = [1, 2]
reverse([]) = []
```

```
sort([3, 2, 1]) = [1, 2, 3]
sort([2, 1]) = [1, 2]
sort([]) = []
```

Functional Specifications

$\forall x, y \in \text{List}[\text{Int}], \text{reverse}(x) = y \Rightarrow$

$$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 1\}, x_i = y_{\text{len}(y)-1-i}$$

1. Input and output has the same length n ;
2. The i -th element in x is the same as $(n - 1 - i)$ -th element in y

```
reverse([3, 2, 1]) = [1, 2, 3]
reverse([2, 1]) = [1, 2]
reverse([]) = []
```

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$$

1. Input and output has the same length n ;
2. In result y , the i -th element is always less than or equal to the $(i + 1)$ -th element

```
sort([3, 2, 1]) = [1, 2, 3]
sort([2, 1]) = [1, 2]
sort([]) = []
```

Functional Specifications

Pre-condition:

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$

Post-condition:

Functional Specifications

Pre-condition:

$x \in \text{List}[\text{Int}]$

x is an integer list

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$

Post-condition:

Functional Specifications

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$

Pre-condition:

$x \in \text{List}[\text{Int}]$

x is an integer list

Post-condition:

$y \in \text{List}[\text{Int}]$

y is an integer list

$\text{len}(x) = \text{len}(y)$

y and x has the same length

$y_{i+1} \geq y_i$

Latter element in y always
bigger than or equal to
previous element

Form: Pre- and Post-Conditions

Premise: All valid inputs to a function **MUST** satisfy

Pre-condition:

$x \in \text{List}[\text{Int}]$

x is an integer list

$\forall x, y \in \text{List}[\text{Int}], \text{sort}(x) = y \Rightarrow$

$\text{len}(x) = \text{len}(y) \wedge \forall i \in \{0 \dots \text{len}(x) - 2\}, y_{i+1} \geq y_i$

Promise: all outputs **WILL** satisfy if the premise holds

Post-condition:

$y \in \text{List}[\text{Int}]$

y is an integer list

$\text{len}(x) = \text{len}(y)$

y and x has the same length

$y_{i+1} \geq y_i$

Latter element in y always bigger than or equal to previous element

Pre- and Post-Conditions in the Wild

Russol (Rust)

```
method Find(a: array<int>, key: int) returns (index: int)
  ensures 0 <= index ==> index < a.Length && a[index] == key
  ensures index < 0 ==> forall k :: 0 <= k < a.Length ==> a[k] != key
{
  index := 0;
  while index < a.Length
  {
    if a[index] == key { return; }
    index := index + 1;
  }
  index := -1;
}
```

Dafny

```
int abs_val(int x)
  _Pre_ true
  _Post_ (retval >= 0)
{
  if (x < 0) {
    return -x;
  } else {
    return x;
  }
}
```

Checked-C

```
int main() {
  int i;
  int j=__VERIFIER_nondet_int();
  int n=__VERIFIER_nondet_int();
  assume_abort_if_not(n < 100000);
  int a[n];

  assume_abort_if_not(j>0 && j < 10000);
  for(i=1;i<n;i++) {
    int k=__VERIFIER_nondet_int();
    assume_abort_if_not(k>0 && k < 10000);
    a[i]=i+j+k;
  }

  for(i=1;i<n;i++)
    __VERIFIER_assert(a[i]>=(i+2));
  return 0;
}
```

```
#[requires(self.len() > 0)]
fn peek(&self) -> &T {
  todo!()
}
```

```
{{ True }}
while X ≠ 0 do
  X := X - 1
end
{{ X = 0 }}
```

Software Foundations /
Hoare Logic (Coq)

SV-Comp (C)

Pre- and Post-Condition Practice

```
def insert(list: List[int], elem: int) -> List[int]:
```

```
def sqrt(x: float) -> float:
```

```
def matrix_mul(a: Tensor, b: Tensor) -> Tensor:
```

Completeness of Pre- and Post-Conditions

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Is this post-condition
complete?

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

Completeness of Pre- and Post-Conditions

Input: [1, 2, 3, 4, 5]

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

Output: [1, 2, 3, 4, 5, 6]

Is this post-condition
complete? **✗**

Completeness of Pre- and Post-Conditions

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$
 $\text{len}(y) = n$

Attempt 2:
Is this post-condition
complete?

Completeness of Pre- and Post-Conditions

Input: [1, 2, 3, 4, 5]

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

$\text{len}(y) = n$

Output: [5, 5, 5, 5, 5]

Attempt 2:
Is this post-condition
complete? **✗**

Completeness of Pre- and Post-Conditions

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Attempt 3:
Is this post-condition
complete?

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

$\text{len}(y) = n$

$\forall i. 0 \leq i < n \Rightarrow \exists j. x[i] = y[j]$

Completeness of Pre- and Post-Conditions

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Attempt 3:
Is this post-condition
complete?

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

$\text{len}(y) = n$

$\forall i. 0 \leq i < n \Rightarrow \exists j. x[i] = y[j]$

$\forall i. 0 \leq i < n \Rightarrow \exists j. y[i] = x[j]$

Completeness of Pre- and Post-Conditions

Input: [1, 2, 3, 4, 2]

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

Output: [1, 2, 3, 4, 4]

$\text{len}(y) = n$

$\forall i. 0 \leq i < n \Rightarrow \exists j. x[i] = y[j]$

$\forall i. 0 \leq i < n \Rightarrow \exists j. y[i] = x[j]$

Attempt 3:
Is this post-condition
complete? **✗**

Completeness of Pre- and Post-Conditions

Pre-condition

$x \in \text{List}[\text{Int}] \quad n = \text{len}(x)$

$y = \text{sort}(x)$

Post-condition

$y \in \text{List}[\text{Int}] \quad \forall i. 0 \leq i < n - 1 \Rightarrow y[i] \leq y[i + 1]$

$\text{len}(y) = n$

$\exists p: \mathbb{Z}_n \rightarrow \mathbb{Z}_n, p \text{ is a permutation,}$
 $\forall i. 0 \leq i < n \Rightarrow y[i] = x[p(i)]$

Attempt 4:
Is this post-condition
complete? 

Difficult to specify complete functional specification!

Leveraging Rust Types for Program Synthesis

JONÁŠ FIALA, ETH Zurich, Switzerland

SHACHAR ITZHAKY, Technion, Israel

PETER MÜLLER, ETH Zurich, Switzerland

NADIA POLIKARPOVA, University of California, San Diego, USA

ILYA SERGEY, National University of Singapore, Singapore

Rust type +
functional spec

```
[#requires ...]  
[#ensures ...]  
fn target(x: T1...) -> T  
  
[#pure]  
fn f(x: T1...) -> T  
...
```

Contributions. In summary, this paper makes the following contributions:

- Synthetic Ownership Logic (SOL), a variant of Separation Logic that is targeted to program synthesis of well-typed Rust programs from type signatures and functional specifications.
- RusSOL, the first synthesizer for Rust code from functional correctness specifications. We built RusSOL by integrating SOL into SuSLik's general-purpose proof search framework.
- An extensive evaluation of RusSOL with regard to utility and performance. We show that it is capable of synthesizing a large number of non-trivial heap-manipulating Rust programs, in a matter of seconds, and that **required annotations are on average 27% shorter than the code.**

Specification itself has a language

- First-order logic operators:
 - $\forall, \exists, =, \neq, \wedge, \vee, \neg, \Rightarrow, \dots$
- Base types, commonly used types, and their predicates
 - `Int`, `Bool`, `Set[T]`, `List[T]`, ...
 - $+, -, \times, \div, ::, \cup, \cap, \in, [..], \&\&, ||, !, len, \dots$
- Logic systems to specify program behaviors
 - Memory (Separation Logic): $\{.\}, \mapsto, *, \dots$
 - Temporal Behavior (Temporal Logic): `Globally`, `Next`, `Finally`, ...
 - Mathematical Objects: `Permutation`, ...

```
{x ↦ a * y ↦ b} void swap(loc x, loc y) {x ↦ b * y ↦ a}
```

Specification itself has a language

Syntax + Semantics

- First-order logic operators:
 - $\forall, \exists, =, \neq, \wedge, \vee, \neg, \Rightarrow, \dots$
- Base types, commonly used types, and their predicates
 - `Int, Bool, Set[T], List[T], ...`
 - `+, -,  $\times$ ,  $\div$ ,  $::$ , U, n, in, len, ...`
- Logic systems to specify program behaviors
 - Memory (Separation Logic): `{.}`, `↪`, `*`, ...
 - Temporal Behavior (Temporal Logic): `Globally`, `Next`, `Finally`, ...
 - Mathematical Objects: `Permutation`, ...

Specification itself has a language

Syntax

+

Semantics

- First-order logic operators:
 - $\forall, \exists, =, \neq, \wedge, \vee, \neg, \Rightarrow, \dots$
- Base types, commonly used types, and their predicates
 - `Int`, `Bool`, `Set[T]`, `List[T]`, ...
 - `+`, `-`, `×`, `÷`, `::`, `U`, `∩`, `∈`, `[.]`, `&&`, `||`, `!`, `len`, ...
- Logic systems to specify program behaviors
 - Memory (Separation Logic): `{.}`, `↦`, `*`, ...
 - Temporal Behavior (Temporal Logic): `Globally`, `Next`, `Finally`, ...
 - Mathematical Objects: `Permutation`, ...

???

Specification itself has a language

Syntax

+

Semantics

- First-order logic operators:
 - $\forall, \exists, =, \neq, \wedge, \vee, \neg, \Rightarrow, \dots$
- Base types, commonly used types, and their predicates
 - $\text{Int}, \text{Bool}, \text{Set}[T], \text{List}[T], \dots$
 - $+, -, \times, \div, ::, \cup, \cap, \in, [..], \&\&, ||, !, \text{len}, \dots$
- Logic systems to specify program behaviors
 - Memory (Separation Logic): $\{.\}, \mapsto, *, \dots$
 - Temporal Behavior (Temporal Logic): Globally, Next, Finally, ...
 - Mathematical Objects: Permutation, ...

$$\{P\} c \{Q\}$$

Hoare Triple

$$[[c]] \models Q$$

Operational Alignment

Verification of Program with a Specification

{Pre-condition} Program {Post-condition}

$$\{P\} c \{Q\}$$

Verification of Program with a Specification

{Pre-condition} Program {Post-condition}

$\{P\} c \{Q\}$

```
{x ↦ a * y ↦ b} void swap(loc x, loc y) {x ↦ b * y ↦ a}
```

```
[#requires ...]  
[#ensures ...]  
fn target(x: T1...) -> T
```

```
VERIFIER_assume(true);  
while (X ≠ 0) {  
  X = X - 1;  
}  
VERIFIER_assert(X = 0);
```

```
{{ True }}  
while X ≠ 0 do  
  X := X - 1  
end  
{{ X = 0 }}
```

Verification of Program with a Specification

{Pre-condition} Program {Post-condition}

$$\{P\} c \{Q\}$$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

P

$x: \text{Int}$

```
int abs(int x) {  
    int y;  
    if (x >= 0) // B1  
        y = x; // B2  
    else  
        y = -x; // B3  
    return y; // B4  
}
```

c

Q

$(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x:Int}  
    int y;  
  
    if (x >= 0) // B1  
  
        y = x; // B2  
  
    else  
  
        y = -x; // B3  
  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer

Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
  
        y = x; // B2  
  
    else  
  
        y = -x; // B3  
  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
  
    else  
  
        y = -x; // B3  
  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
        {x: Int, y: Int, x ≥ 0, x = y}  
    else  
  
        y = -x; // B3  
  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
        {x: Int, y: Int, x ≥ 0, x = y}  
    else  
        {x: Int, y: Int, x < 0}  
        y = -x; // B3  
  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
        {x: Int, y: Int, x ≥ 0, x = y}  
    else  
        {x: Int, y: Int, x < 0}  
        y = -x; // B3  
        {x: Int, y: Int, x < 0, y = -x}  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
        {x: Int, y: Int, x ≥ 0, x = y}  
    else  
        {x: Int, y: Int, x < 0}  
        y = -x; // B3  
        {x: Int, y: Int, x < 0, y = -x}  
    {x: Int, y: Int, (x < 0 ∧ y = -x) ∨ (x ≥ 0 ∧ x = y)}  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
        {x: Int, y: Int, x ≥ 0, x = y}  
    else  
        {x: Int, y: Int, x < 0}  
        y = -x; // B3  
        {x: Int, y: Int, x < 0, y = -x}  
    {x: Int, y: Int, (x < 0 ∧ y = -x) ∨ (x ≥ 0 ∧ x = y)}  
    return y; // B4  
}
```

Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
    {x: Int}  
    int y;  
    {x: Int, y: Int}  
    if (x >= 0) // B1  
        {x: Int, y: Int, x ≥ 0}  
        y = x; // B2  
        {x: Int, y: Int, x ≥ 0, x = y}  
    else  
        {x: Int, y: Int, x < 0}  
        y = -x; // B3  
        {x: Int, y: Int, x < 0, y = -x}  
  
    return y; // B4  
}
```

$\{x: \text{Int}, y: \text{Int}, (x < 0 \wedge y = -x) \vee (x \geq 0 \wedge x = y)\}$



Goal: $(y = -x \vee y = x) \wedge y \geq 0$

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
  {x: Int}  
  int y;  
  {x: Int, y: Int}  
  if (x >= 0) // B1  
    {x: Int, y: Int, x ≥ 0}  
    y = x; // B2  
    {x: Int, y: Int, x ≥ 0, x = y}  
  else  
    {x: Int, y: Int, x < 0}  
    y = -x; // B3  
    {x: Int, y: Int, x < 0, y = -x}  
  
  return y; // B4  
}
```

$\{x: \text{Int}, y: \text{Int}, (x < 0 \wedge y = -x) \vee (x \geq 0 \wedge x = y)\}$



Goal: $(y = -x \vee y = x) \wedge y \geq 0$

?

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
  {x: Int}  
  int y;  
  {x: Int, y: Int}  
  if (x >= 0) // B1  
    {x: Int, y: Int, x ≥ 0}  
    y = x; // B2  
    {x: Int, y: Int, x ≥ 0, x = y}  
  else  
    {x: Int, y: Int, x < 0}  
    y = -x; // B3  
    {x: Int, y: Int, x < 0, y = -x}  
  
  return y; // B4  
}
```

$\{x: \text{Int}, y: \text{Int}, (x < 0 \wedge y = -x) \vee (x \geq 0 \wedge x = y)\}$



Goal: $(y = -x \vee y = x) \wedge y \geq 0$



Z3 Theorem Prover!

Can you write a python program with Z3 to check the satisfiability of the following

$\{x:\text{Int}, y:\text{Int}, (x < 0 \wedge y = -x) \vee (x \geq 0 \wedge x = y)\} \Rightarrow \text{Goal: } (y = -x \vee y = x) \wedge y \geq 0$

Thought for 16s

Here's a tiny Z3Py script that checks **validity** of your implication by asking Z3 to find a counterexample to `pre ∧ ¬goal`. If that is **unsat**, then the implication holds for all integers `x, y`.

python

 Copy code

```
# pip install z3-solver (if not already installed)
from z3 import *

# Vars
x, y = Ints('x y')

# Specification
pre = Or(And(x < 0, y == -x),
         And(x >= 0, x == y))

goal = And(Or(y == -x, y == x),
           y >= 0)

# Check validity by searching for a counterexample to pre → goal
s = Solver()
s.add(pre, Not(goal))
res = s.check()
```

Program c : a program state transformer
Pre- and Post-condition P, Q : a Boolean function over program states

$$\forall x, P(x) \Rightarrow Q(c(x))$$

```
int abs(int x) {  
  {x: Int}  
  int y;  
  {x: Int, y: Int}  
  if (x >= 0) // B1  
    {x: Int, y: Int, x ≥ 0}  
    y = x; // B2  
    {x: Int, y: Int, x ≥ 0, x = y}  
  else  
    {x: Int, y: Int, x < 0}  
    y = -x; // B3  
    {x: Int, y: Int, x < 0, y = -x}  
  
  return y; // B4  
}
```

$\{x: \text{Int}, y: \text{Int}, (x < 0 \wedge y = -x) \vee (x \geq 0 \wedge x = y)\}$



Goal: $(y = -x \vee y = x) \wedge y \geq 0$



Z3 Theorem Prover!



Software Foundations: Hoare Logic (Coq)

```

{{ True }} =>
{{ m = m }}
X := m

                                {{ X = m }} =>
                                {{ X = m ∧ p = p }};

Z := p;

                                {{ X = m ∧ Z = p }} =>
                                {{ Z - X = p - m }}

while X ≠ 0 do

                                {{ Z - X = p - m ∧ X ≠ 0 }} =>
                                {{ (Z - 1) - (X - 1) = p - m }}

    Z := Z - 1

                                {{ Z - (X - 1) = p - m }};

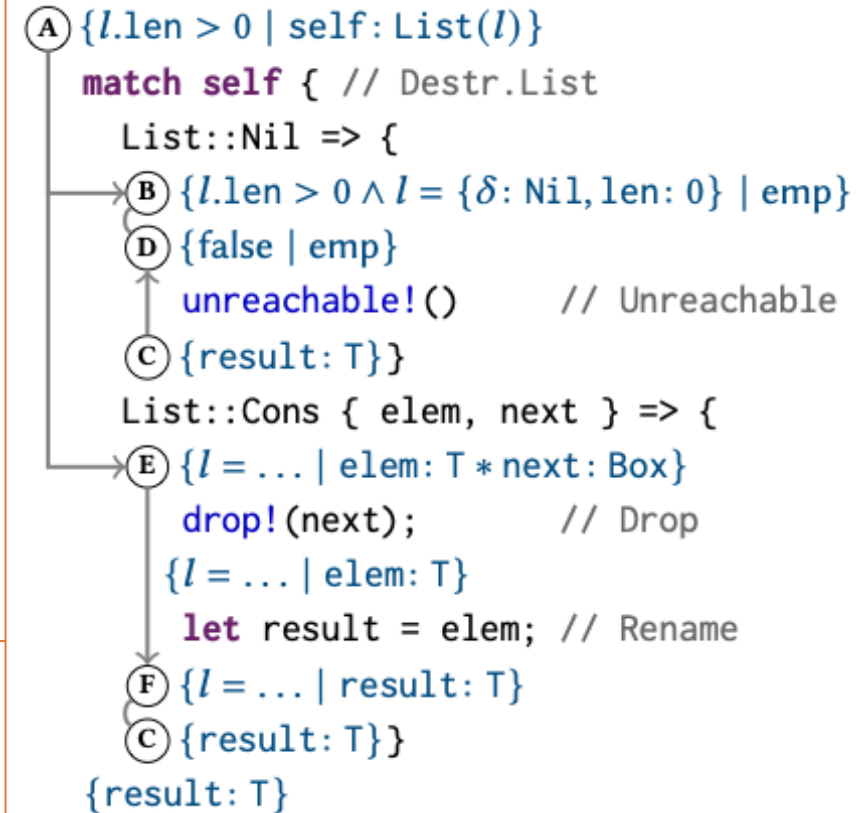
    X := X - 1

                                {{ Z - X = p - m }}

end

{{ Z - X = p - m ∧ ¬ (X ≠ 0) }} =>
{{ Z = p - m }}

```



Leveraging Rust Types for Program Synthesis (Fiala et. al., 2023)



Z3 Theorem Prover!

Satisfiability Modulo Theories (SMT) Solver

Others: CVC3, CVC4, CVC5, Beaver, UCLID, veriT, ...

$$\{P\} c \{Q\}$$

Proof-Theoretic Semantics



Z3 Theorem Prover!

Satisfiability Modulo Theories (SMT) Solver

Others: CVC3, CVC4, CVC5, Beaver, UCLID, veriT, ...

Specification language

Syntax

+

Semantics

- First-order logic operators:
 - $\forall, \exists, =, \neq, \wedge, \vee, \neg, \Rightarrow, \dots$
- Base types, commonly used types, and their predicates
 - `Int`, `Bool`, `Set[T]`, `List[T]`, ...
 - `+`, `-`, `×`, `÷`, `::`, `U`, `∩`, `∈`, `[. ]`, `&&`, `||`, `!`, `len`, ...
- Logic systems to specify program behaviors
 - Memory (Separation Logic): `{. }`, `↪`, `*`, ...
 - Temporal Behavior (Temporal Logic): `Globally`, `Next`, `Finally`, ...
 - Mathematical Objects: `Permutation`, ...

$$\{P\} c \{Q\}$$

Hoare Triple



Functional Specification Guided Synthesis

Structural Specification

- Target Language
- Syntax & Semantics
- Types

Functional Specification Guided Synthesis

Structural Specification

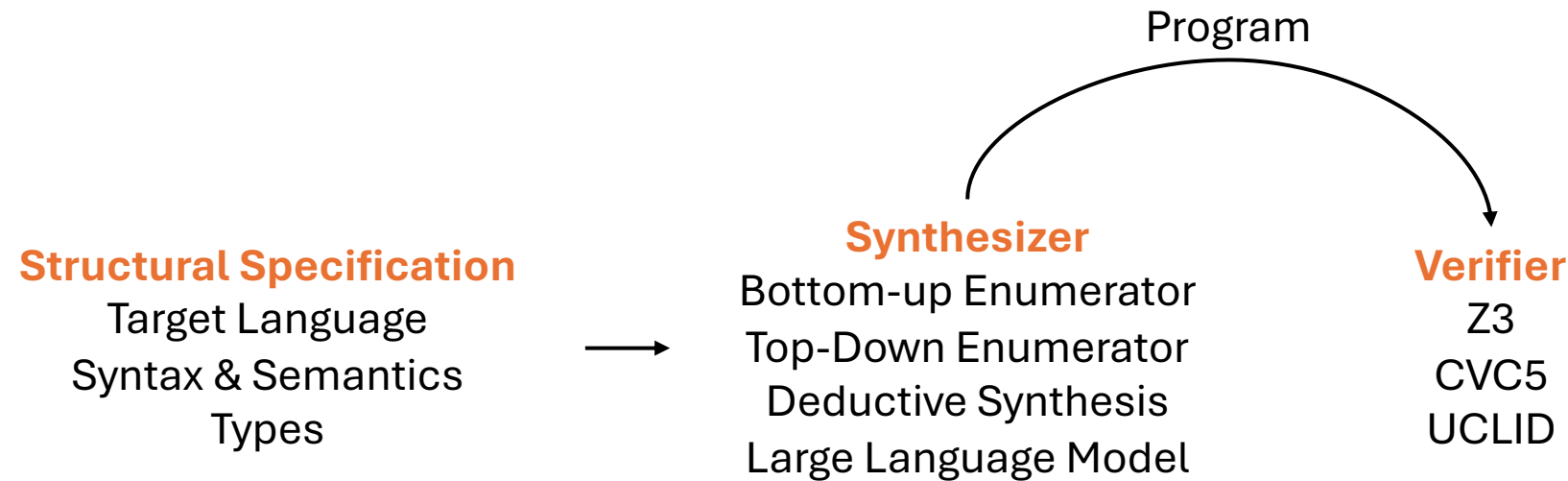
Target Language
Syntax & Semantics
Types



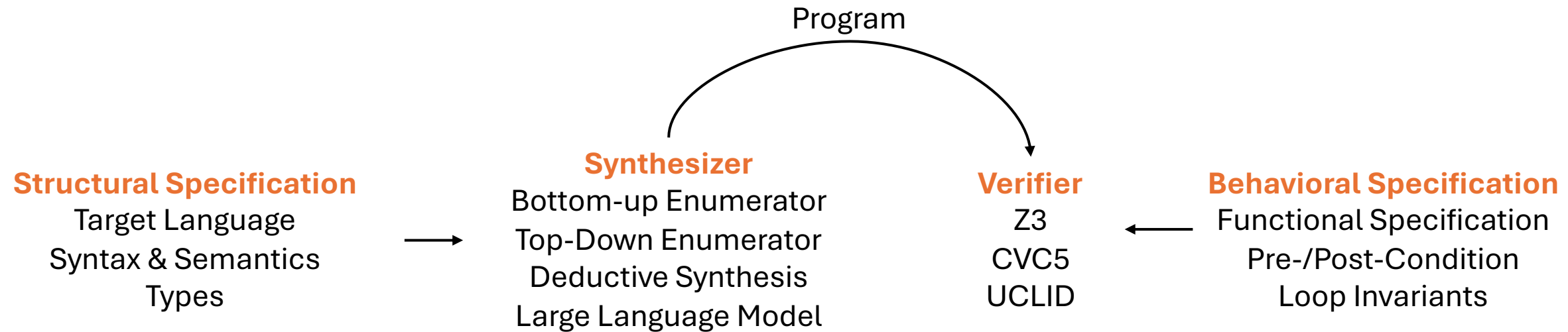
Synthesizer

Bottom-up Enumerator
Top-Down Enumerator
Deductive Synthesis
Large Language Model

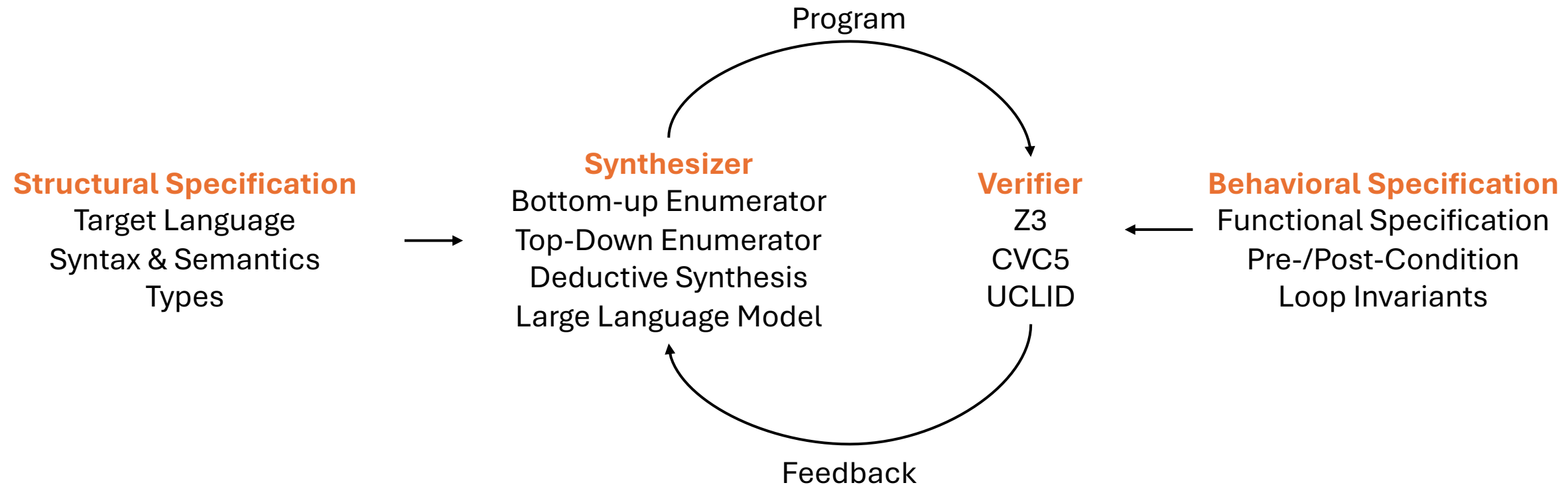
Functional Specification Guided Synthesis



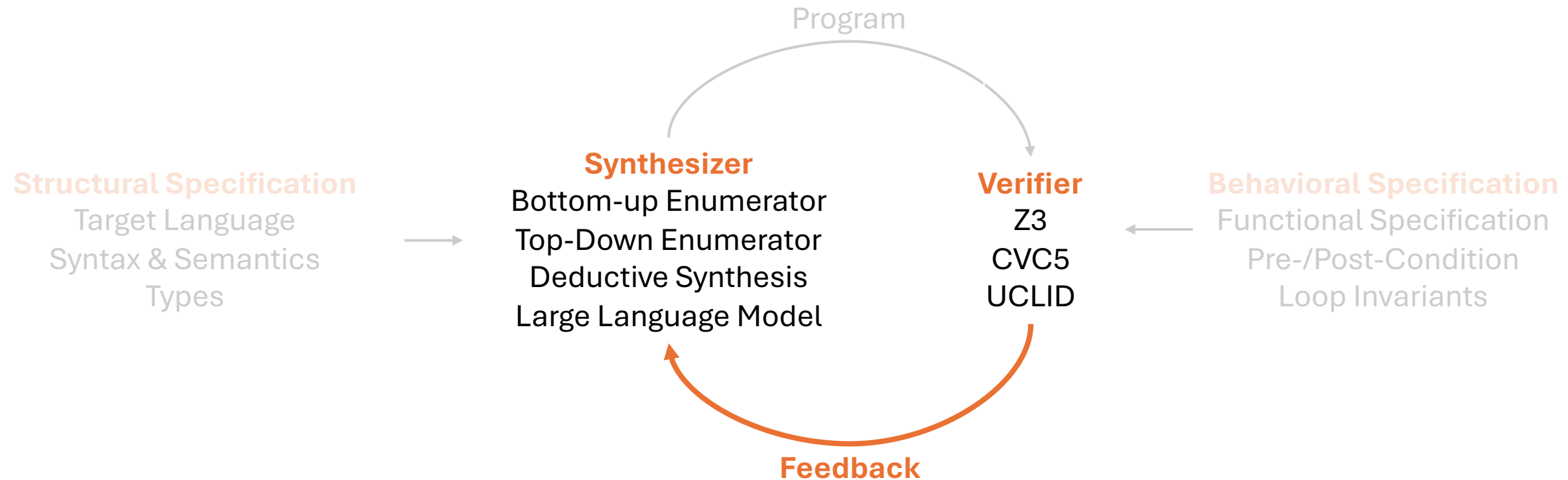
Functional Specification Guided Synthesis



Functional Specification Guided Synthesis



Functional Specification Guided Synthesis



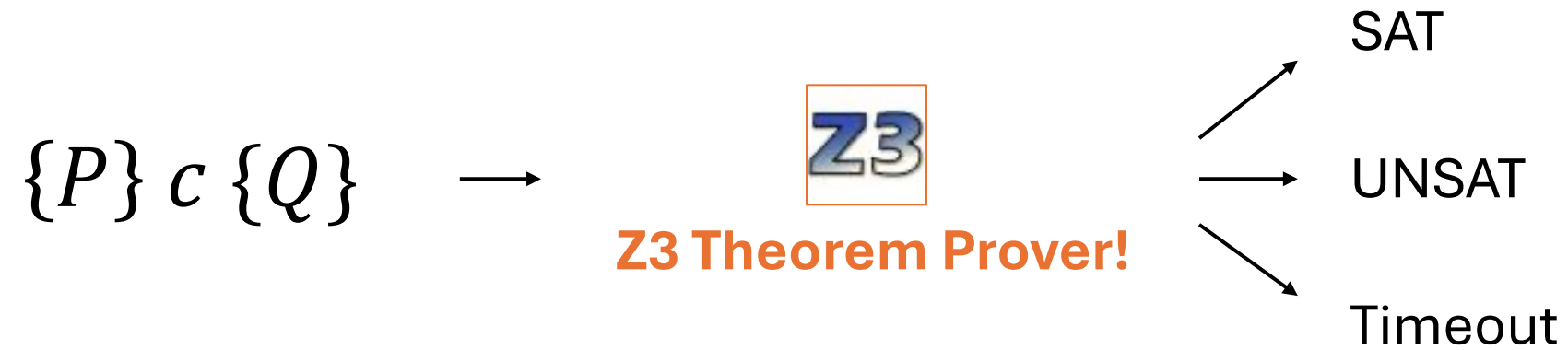
Verifier's Feedback

$$\{P\} \ c \ \{Q\}$$

Verifier's Feedback

$\{P\} c \{Q\} \longrightarrow$ 
Z3 Theorem Prover!

Verifier's Feedback



Verifier's Feedback

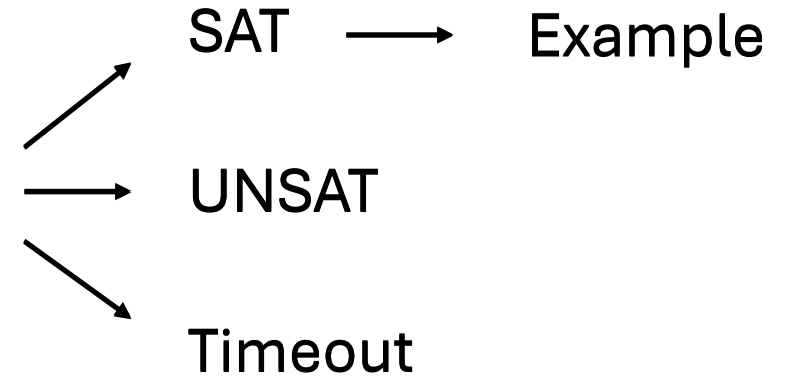


Verifier's Feedback

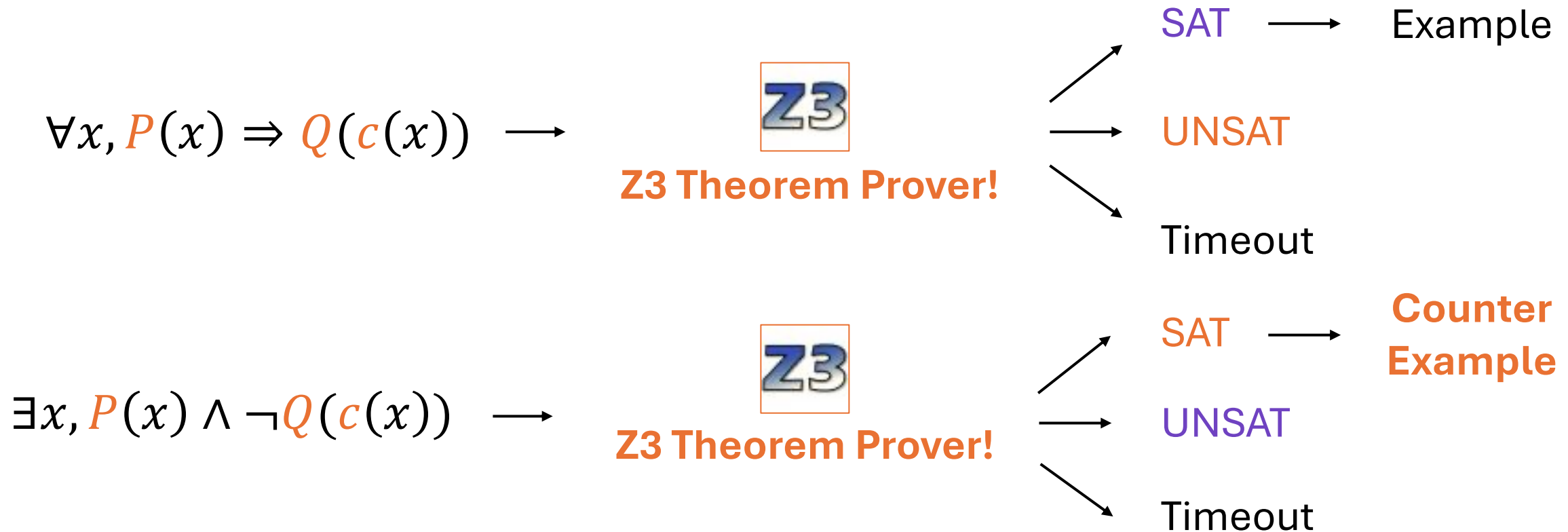
$\forall x, P(x) \Rightarrow Q(c(x)) \rightarrow$



Z3 Theorem Prover!



Verifier's Feedback



```

x, y = Ints('x y')

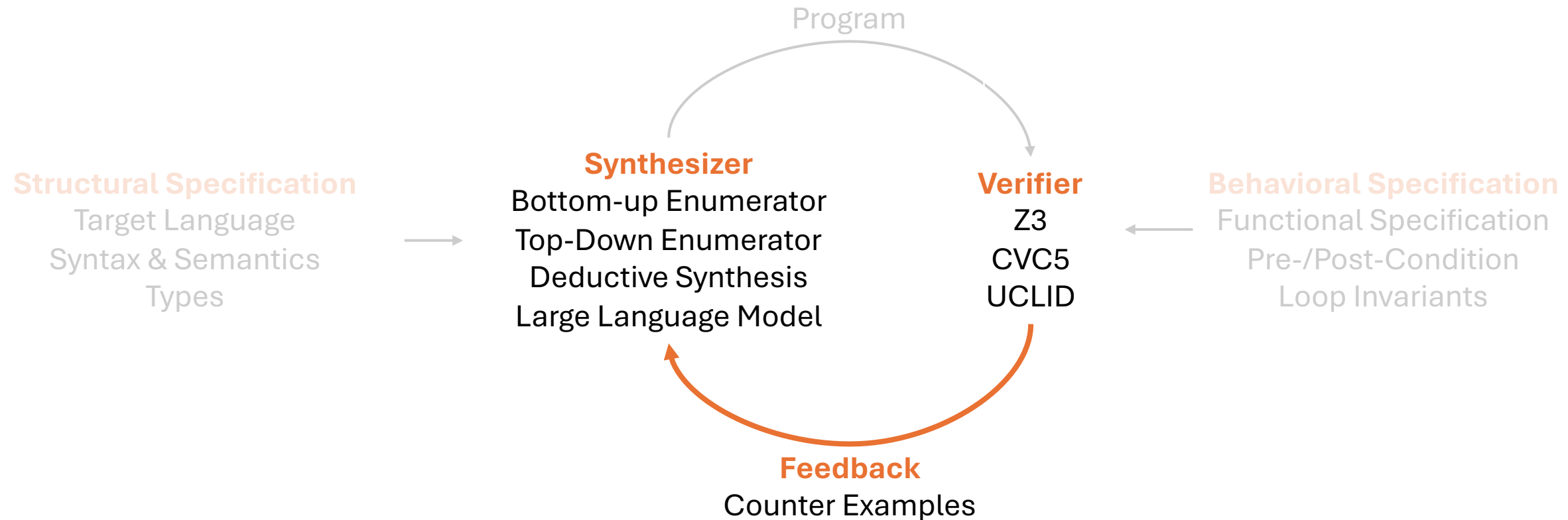
# Original pre and goal
pre  = Or(And(x < 0, y == -x),
          And(x >= 0, x == y))

goal = And(Or(y == -x, y == x),
           y >= 0)

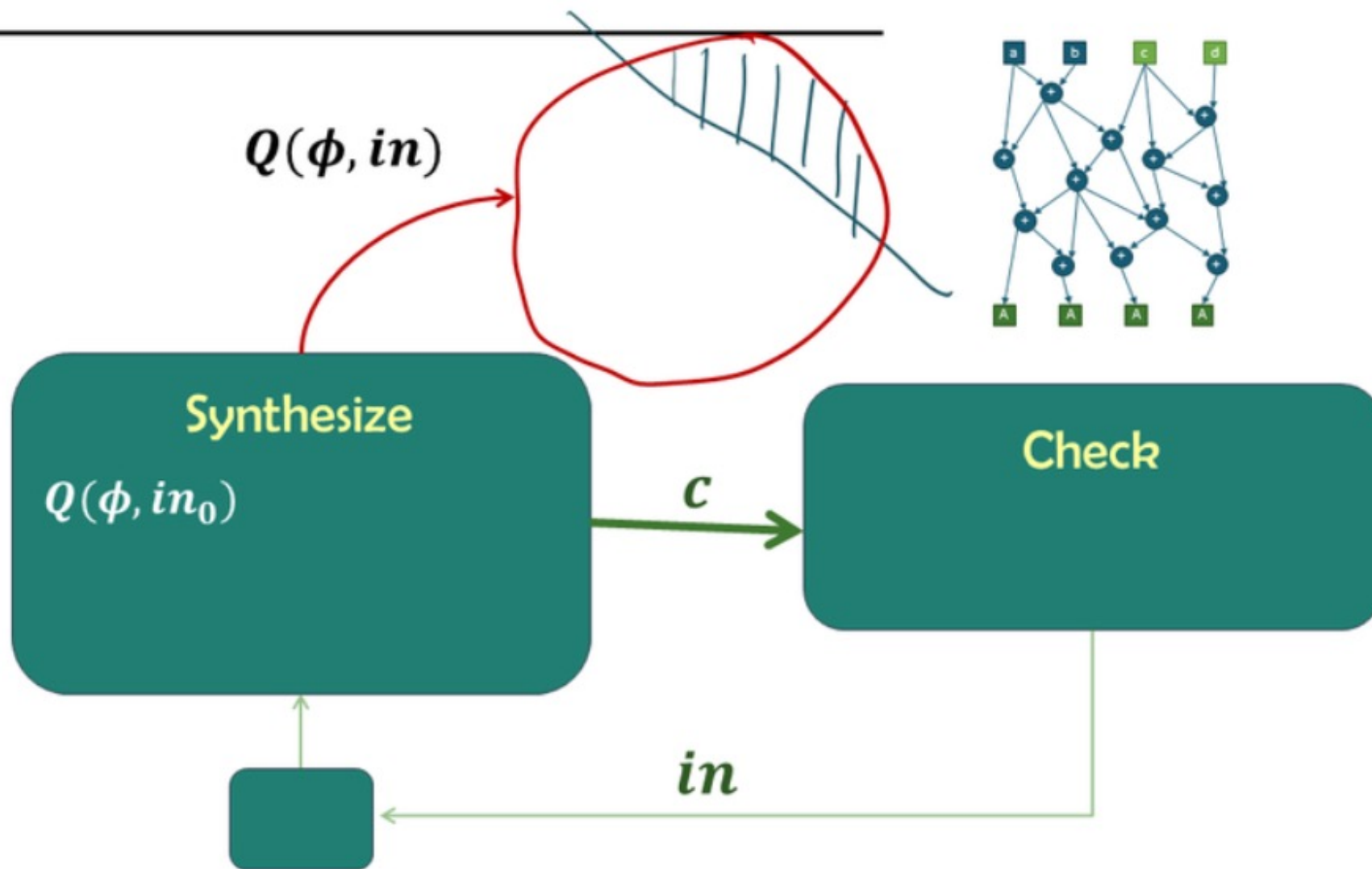
# 1) Try to find a counterexample to the original implication
s = Solver()
s.add(pre, Not(goal))
r = s.check()
print("Counterexample to original (pre → goal)?", r)
if r == sat:
    m = s.model()
    print("Counterexample model:", "x =", m[x], ", y =", m[y])
else:
    print("No counterexample; implication is VALID.")

```

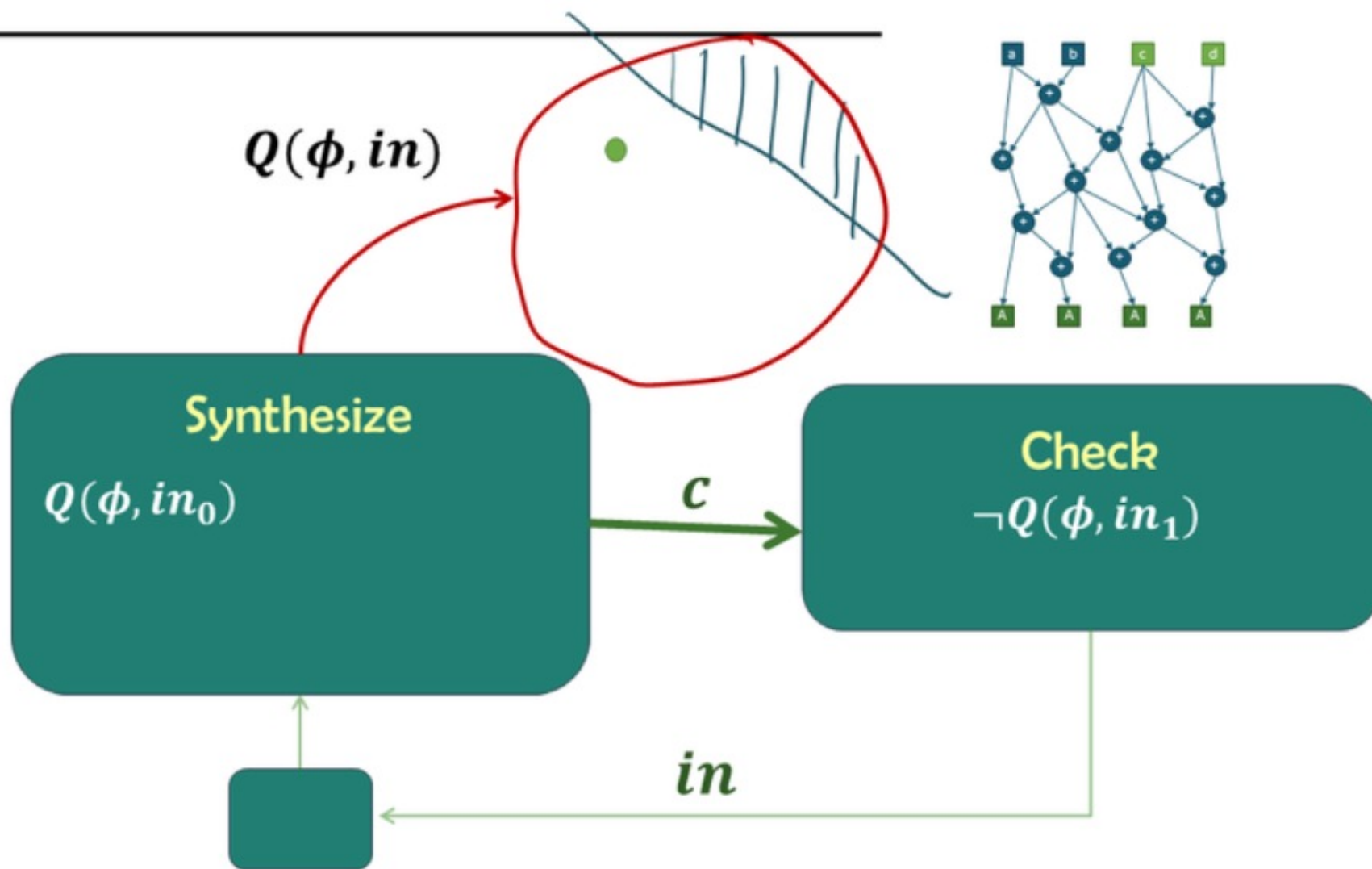
Functional Specification Guided Synthesis



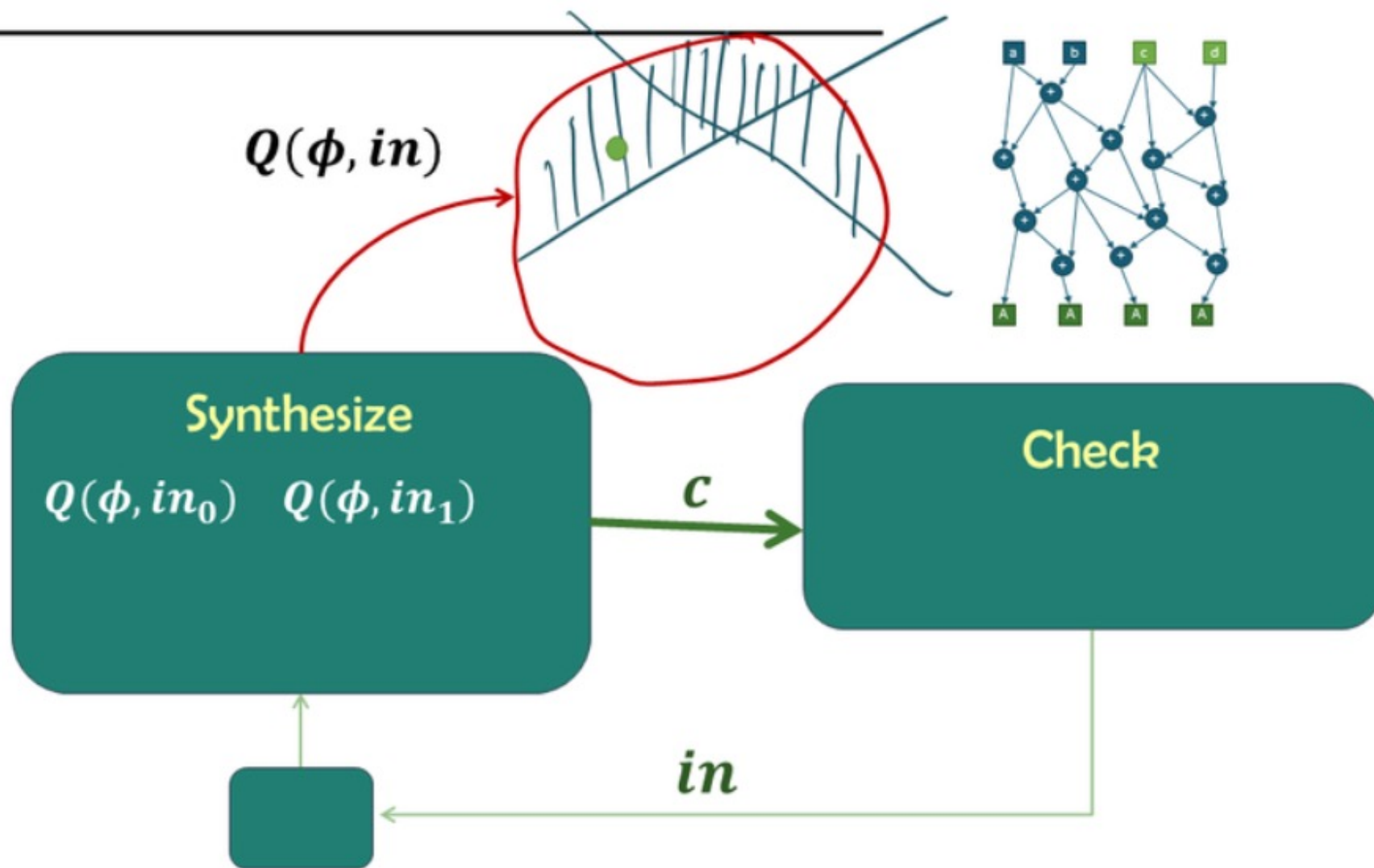
Constraint-based CEGIS



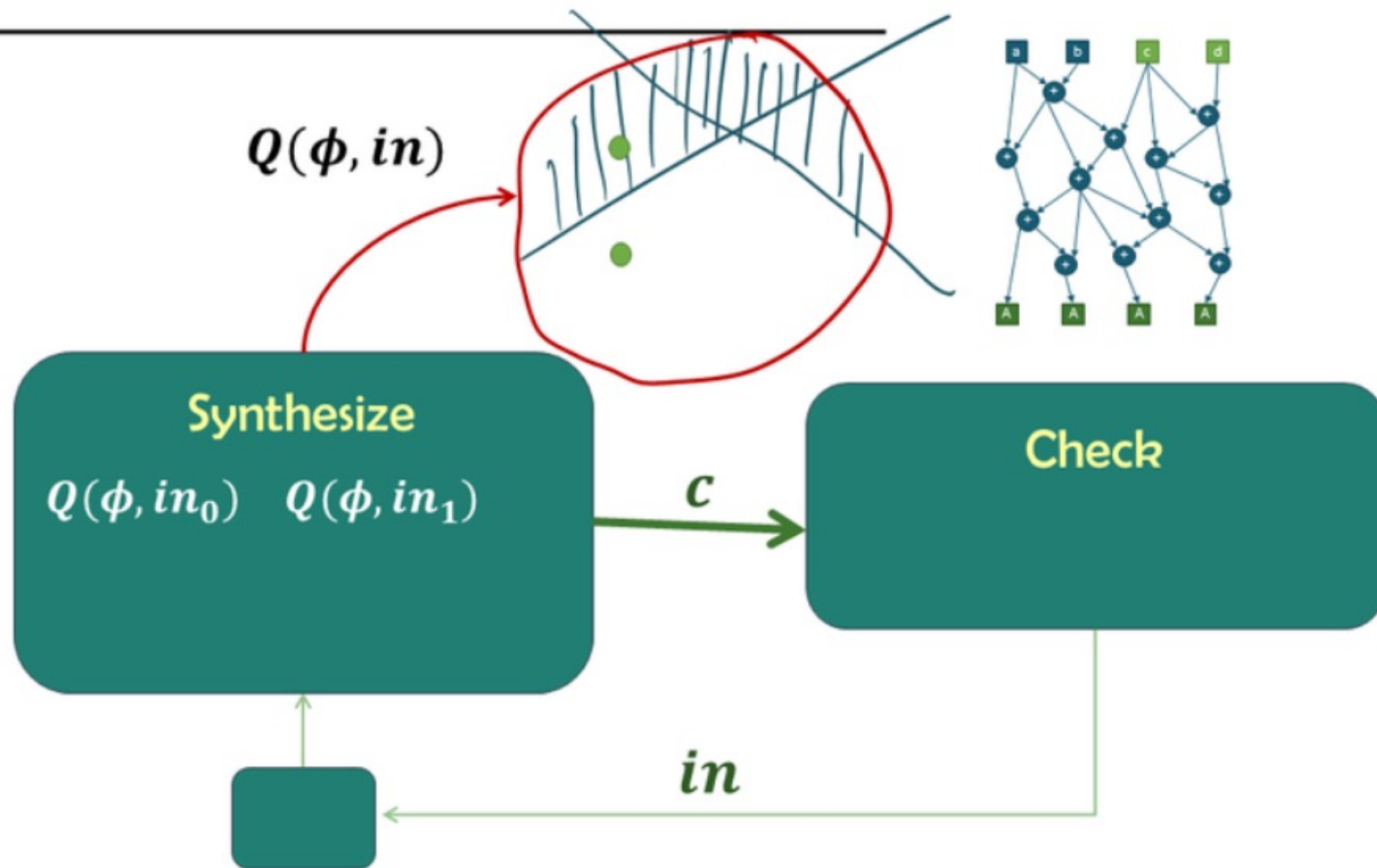
Constraint-based CEGIS



Constraint-based CEGIS



Constraint-based CEGIS



Notes

- There is **NO FREE LUNCH!**
 - Where do you get the functional specifications?
 - Where do you get the language for functional specifications? Is it expressive enough to model program behaviors?
 - Is your functional specification complete?
 - How do you deal with Loops?
 - Loops need “Loop Invariants” for an automated theorem prover to function
 - How do you get the loop invariants?
 - Z3: How long does it take for Z3 to verify $\{P\} \text{ c } \{Q\}$?
 - Often time, Z3 cannot deal with extremely large programs, and will timeout
 - Who is doing the simplification?
 - Verification $\leftrightarrow$ Synthesis interplay overhead and deficiency?

Notes

Specification: The Biggest Bottleneck in Formal Methods and Autonomy

Kristin Yvonne Rozier^(✉)

Iowa State University, Ames, IA, USA
kyrozier@iastate.edu

- There is **NO FREE LUNCH!**
 - Where do you get the functional specifications?
 - Where do you get the language for functional specifications? Is it expressive enough to model program behaviors?
 - Is your functional specification complete?
 - How do you deal with Loops?
 - Loops need “Loop Invariants” for an automated theorem prover to function
 - How do you get the loop invariants?
 - Z3: How long does it take for Z3 to verify $\{P\} \text{ c } \{Q\}$?
 - Often time, Z3 cannot deal with extremely large programs, and will timeout
 - Who is doing the simplification?
 - Verification $\leftrightarrow$ Synthesis interplay overhead and deficiency?

seL4: Formal Verification of an OS Kernel

Gerwin Klein^{1,2}, Kevin Elphinstone^{1,2}, Gernot Heiser^{1,2,3}

June Andronick^{1,2}, David Cock¹, Philip Derrin^{1*}, Dhammika Elkaduwe^{1,2†}, Kai Engelhardt^{1,2}
Rafal Kolanski^{1,2}, Michael Norrish^{1,4}, Thomas Sewell¹, Harvey Tuch^{1,2†}, Simon Winwood^{1,2}

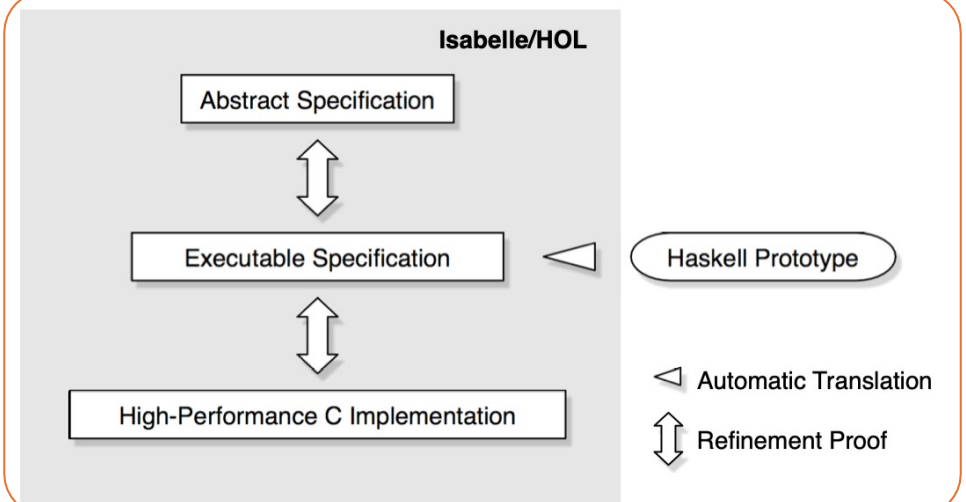
¹ NICTA, ² UNSW, ³ Open Kernel Labs, ⁴ ANU
ertos@nicta.com.au

```
schedule ≡ do
  threads ← all_active_tcbs;
  thread ← select threads;
  switch_to_thread thread
od OR switch_to_idle_thread
```

Figure 3: Isabelle/HOL code for scheduler at abstract level.

	Haskell/C LOC	Isabelle LOC	Invariants	Proof LOP
abst.	—	4,900	~ 75	110,000
exec.	5,700	13,000	~ 80	55,000
impl.	8,700	15,000	0	

Table 1: Code and proof statistics.



IronFleet: Proving Practical Distributed Systems Correct

Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch,
Bryan Parno, Michael L. Roberts, Srinath Setty, Brian Zill

Microsoft Research

```
method Main() {
  var s := ImplInit();
  while (true)
    invariant ImplInvariant(s);
  {
    ghost var journal_old := get_event_journal();
    ghost var ios_performed: seq<IoEvent>;
    s, ios_performed := ImplNext(s);
    assert get_event_journal() ==
      journal_old + ios_performed;
    assert ReductionObligation(ios_performed);
  }
}
```

Figure 8. Mandatory host event-handler loop.

	Spec (source lines of code)	Impl	Proof	Time to Verify (minutes)
High-Level Spec:				
IronRSL	85	–	–	–
IronKV	34	–	–	–
Temporal Logic	208	–	–	–
Distributed Protocol:				
IronRSL Protocol	–	–	1202	4
Refinement	35	–	3379	26
Liveness	167	–	7869	115
IronKV Protocol	–	–	726	2
Refinement	36	–	3998	12
Liveness	98	–	2093	23
TLA Library	–	–	1824	2
Implementation:				
IO/Native Interface	591	–	–	–
Common Libraries	134	833	7690	13
IronRSL	6	2941	7535	152
IronKV	6	1340	2937	42
Total	1400	5114	39253	395

Figure 12. Code sizes and verification times.

Notes

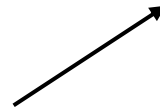
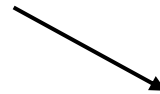
- “Multi-Modal” Program Synthesis

Structural Specification

Target Language
Syntax & Semantics
Partial Program; Templates
...

Behavioral Specification

Examples
Counter Examples
Functional Specifications
Refinement Types
Natural Language
...



Synthesizer

Bottom-up Enumerator
Top-Down Enumerator
Deductive Synthesis
Large Language Model

Notes

- “Multi-Modal” Program Synthesis

Structural Specification

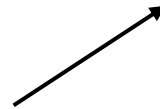
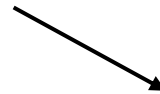
Target Language
Syntax & Semantics
Partial Program; Templates
...

Behavioral Specification

Examples
Counter Examples
Functional Specifications
Refinement Types

Natural Language

...



Synthesizer

Bottom-up Enumerator
Top-Down Enumerator
Deductive Synthesis
Large Language Model

Notes

- Refinement Type as Functional Specification:

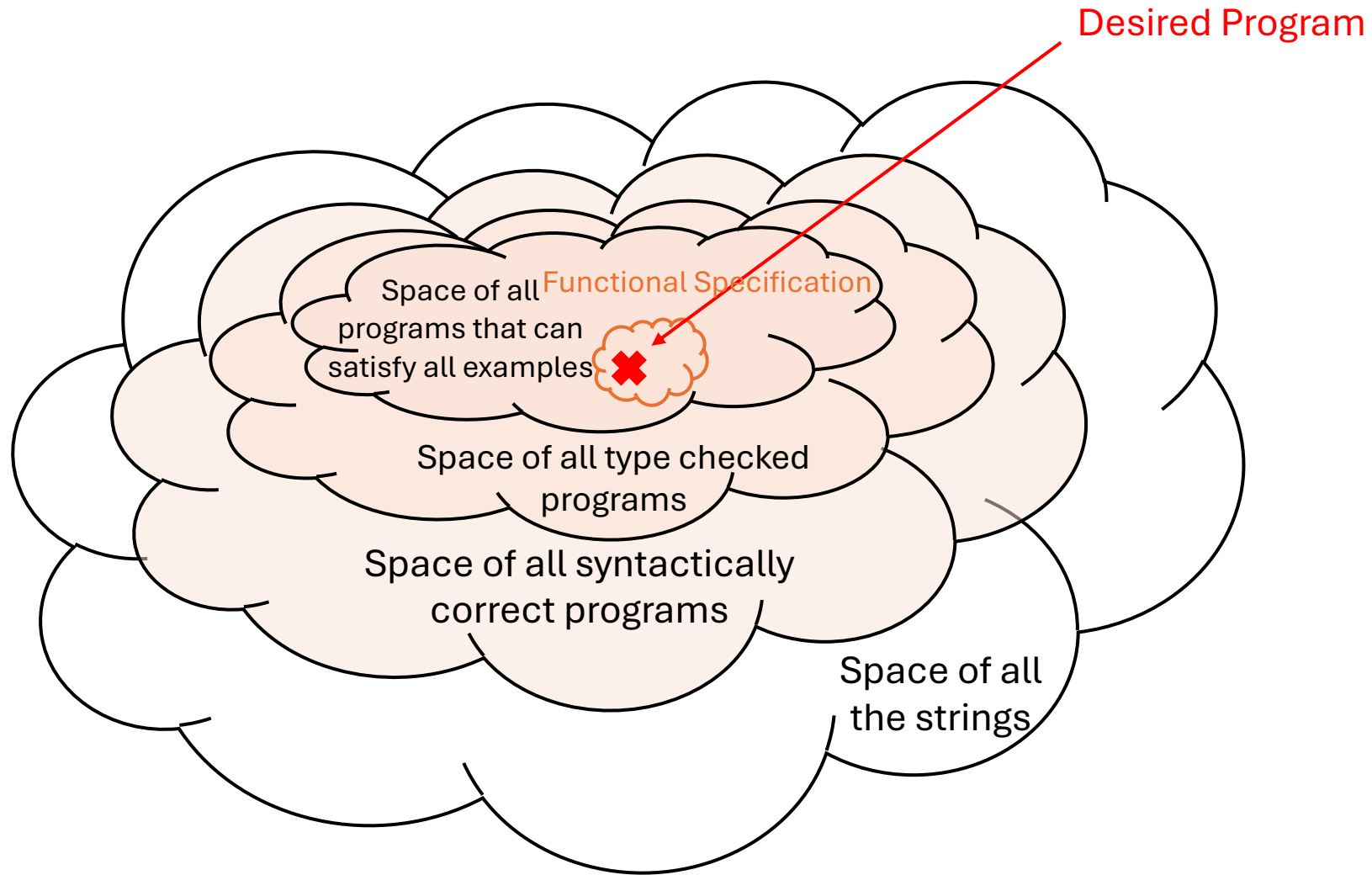
```
def abs(x: int) -> int:
```

```
...
```

```
def abs(x: int) -> {v: int | v >= 0}:
```

```
...
```

High Level Picture

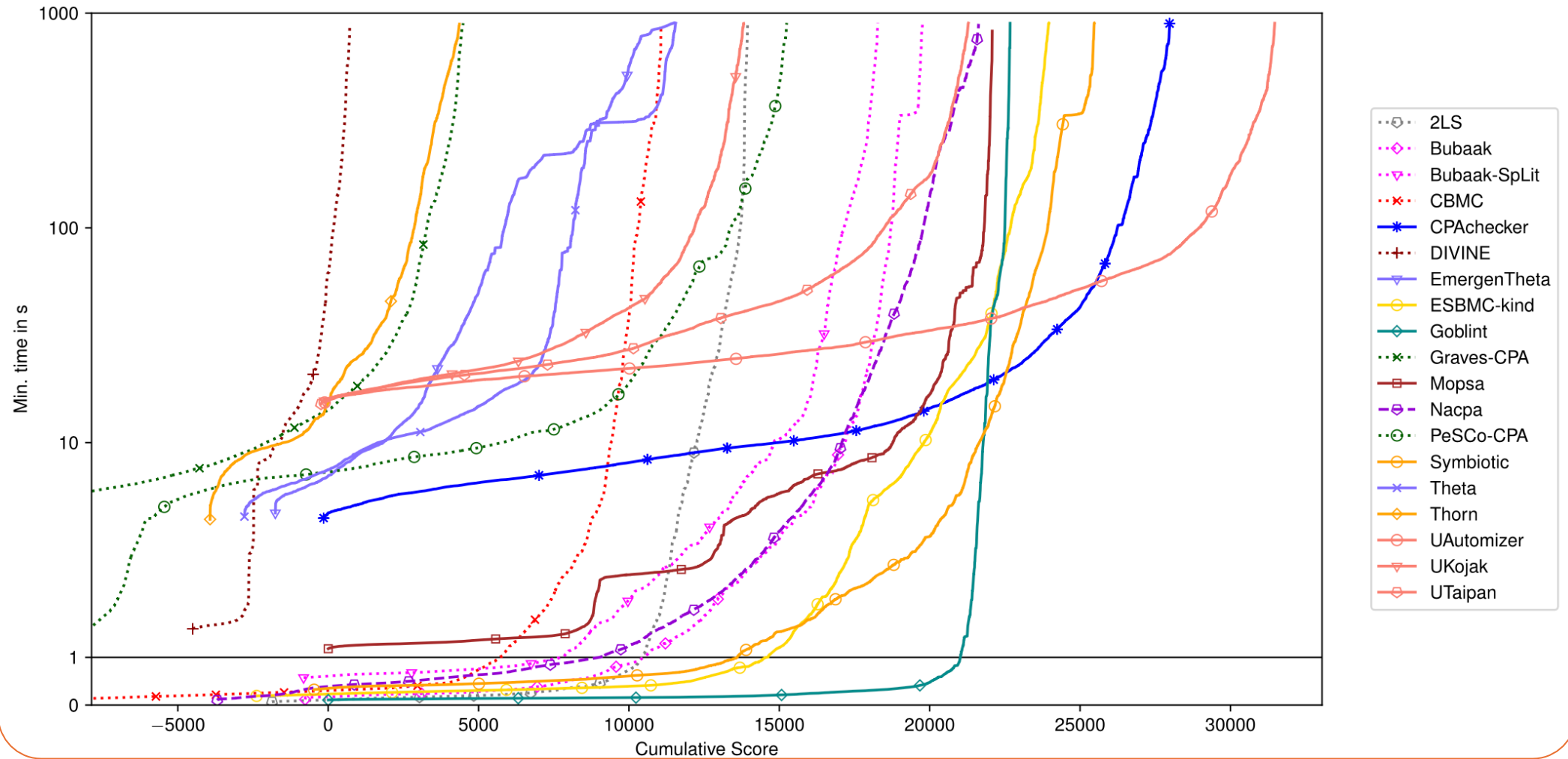


C.Overall

1. UAutomizer

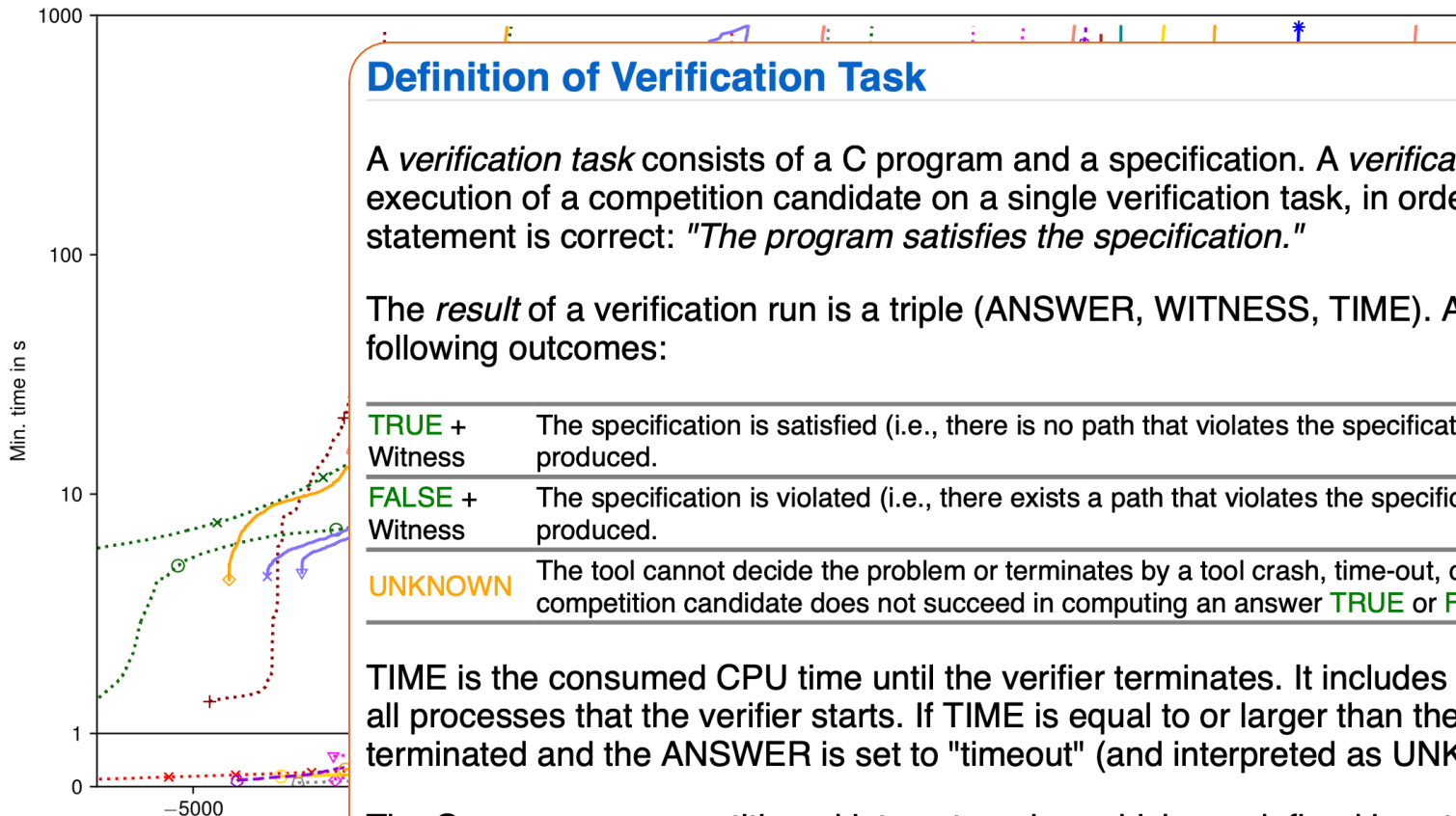
2. CPAchecker

3. Symbiotic



C.Overall

1. UAutomizer
2. CPAchecker
3. Symbiotic



Definition of Verification Task

A *verification task* consists of a C program and a specification. A *verification run* is a non-interactive execution of a competition candidate on a single verification task, in order to check if the following statement is correct: "*The program satisfies the specification.*"

The *result* of a verification run is a triple (ANSWER, WITNESS, TIME). ANSWER is one of the following outcomes:

TRUE + Witness	The specification is satisfied (i.e., there is no path that violates the specification) and a correctness witness is produced.
FALSE + Witness	The specification is violated (i.e., there exists a path that violates the specification) and a violation witness is produced.
UNKNOWN	The tool cannot decide the problem or terminates by a tool crash, time-out, or out-of-memory (i.e., the competition candidate does not succeed in computing an answer TRUE or FALSE).

TIME is the consumed CPU time until the verifier terminates. It includes the consumed CPU time of all processes that the verifier starts. If TIME is equal to or larger than the time limit, then the verifier is terminated and the ANSWER is set to "timeout" (and interpreted as UNKNOWN).

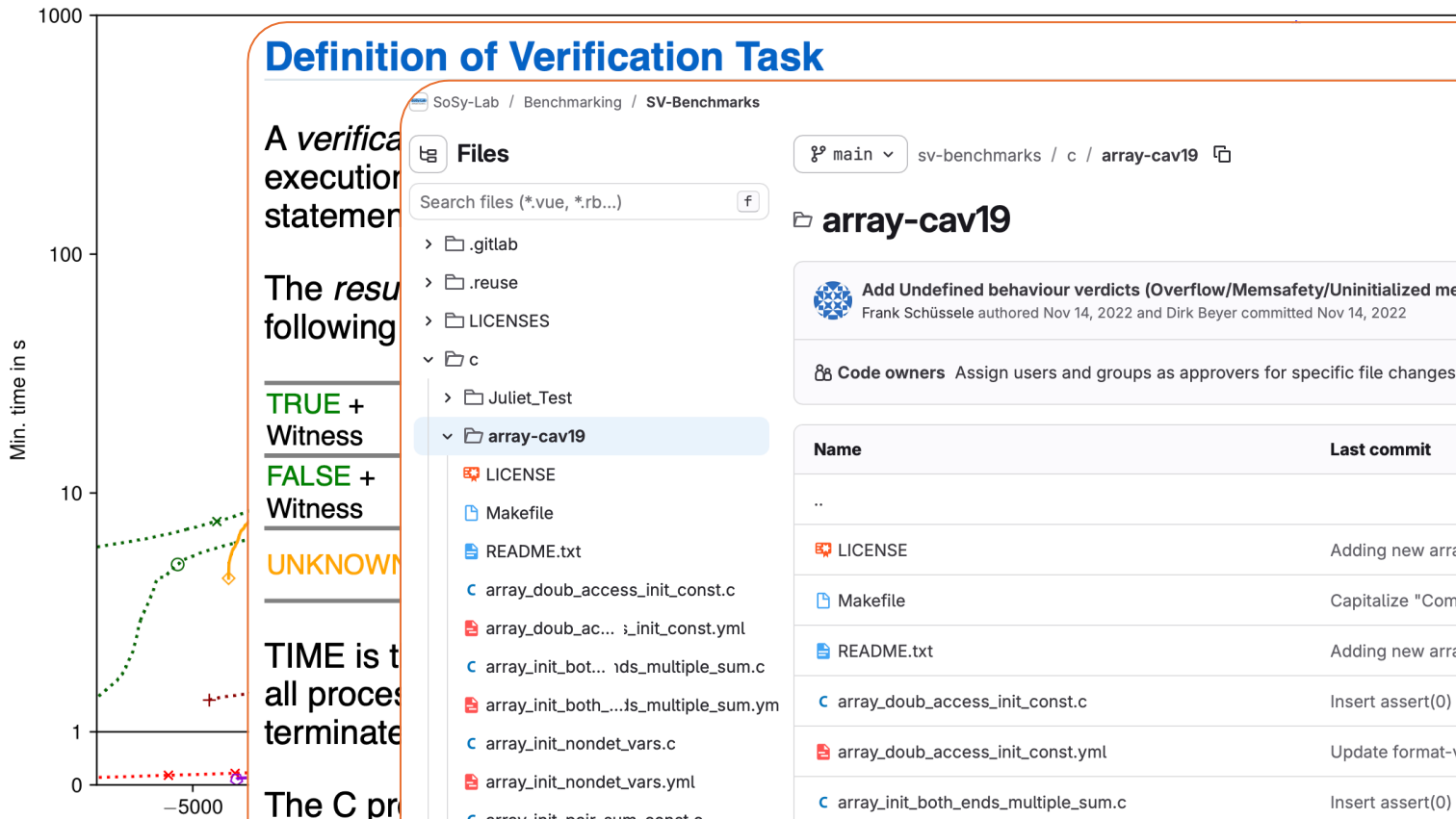
The C programs are partitioned into categories, which are defined in category-set files. The categories and the contained programs are explained on the [benchmark page](#). The requirements for the C programs are described [below in the next section](#).

C.Overall

1. UAutomizer

2. CPAchecker

3. Symbiotic



Definition of Verification Task

A verification task is defined by a set of execution statements and a set of verification goals.

The result of a verification task is a verdict (TRUE, FALSE, UNKNOWN) and a set of witnesses.

TRUE + Witness

FALSE + Witness

UNKNOWN

TIME is the time taken by all processes to terminate.

The C program is categorized into the C program categories.

SoSy-Lab / Benchmarking / SV-Benchmarks

Files

Search files (*.vue, *.rb...)

main sv-benchmarks / c / array-cav19

array-cav19

Add Undefined behaviour verdicts (Overflow/Memsafety/Uninitialized memory)
Frank Schüssele authored Nov 14, 2022 and Dirk Beyer committed Nov 14, 2022

c9415260 History

Code owners Assign users and groups as approvers for specific file changes. [Learn more.](#)

Name	Last commit	Last update
..		
LICENSE	Adding new array and loops reachsafety programs.	6 years ago
Makefile	Capitalize "Community" in copyright notices	5 years ago
README.txt	Adding new array and loops reachsafety programs.	6 years ago
array_doub_access_init_const.c	Insert assert(0) into the body of reach_error	5 years ago
array_doub_access_init_const.yml	Update format-version from 1.1 to 2.0	5 years ago
array_init_both_ends_multiple_sum.c	Insert assert(0) into the body of reach_error	5 years ago
array_init_both_ends_multiple_sum.yml	Update format-version from 1.1 to 2.0	5 years ago
array_init_nondet_vars.c	Insert assert(0) into the body of reach_error	5 years ago
array_init_nondet_vars.yml	Add Undefined behaviour verdicts (Overflow/Memsafety/U...	3 years ago
array_init_pair_sum_const.c	Insert assert(0) into the body of reach_error	5 years ago
array_init_pair_sum_const.yml	Update format-version from 1.1 to 2.0	5 years ago

Week 2

- Assignment 1
 - <https://github.com/machine-programming/assignment-1>
 - Autograder Up on GradeScope
 - There is OOM issue on GradeScope that you need to manage!
 - Due Date Sep 22 (Tuesday)